

C221129
Tamilir

C++ previous Ques. C221129
Spring 22'

① @ Define operator overloading? Write the rules of operator overloading.

⇒ Operator overloading is a process that where different operators have different implementation depending on their arguments, or to assign more than one operation on same operator known as operator overloading.

To achieve operator overloading we have to write a special function known as "operator ()".

Rules / ~~def~~ syntax

```
class class_Name
{
    public:
    return_Type operator symbol (arguments)
    {
        body ...
    }
};
```

Here, (same as class name)
return_Type = return of the function
symbol = + / - / ++ / < etc

```

1(b)
#include <bits/stdc++.h>
using namespace std;

class Float
{
    float a;
public:
    Float(float a) : a(a) {}
    { this -> a = a; }
    float operator + (Float f)
    { return (a + f.a); }
    float operator - (Float f)
    { return (a - f.a); }
};

int main()
{
    Float f1(10.5), f2(20.10);
    cout << "output of + operator" << f1 + f2;
    cout << endl << "output of - operator" << f1 - f2;
    return 0;
}

```

1(c) A friend function can not be used to overload the assignment operator (=) Explain why?

Ans → Basically a friend function can access or use the functions and the data member of a class.

We can overload the assignment operator (=) just by using member function of a class.

But a friend function is a non-member function of a class to which it has been defined as friend. Therefore it just uses the functionality (functions & data) of the class. So it does not consist the implementation for the class.

That's why it can not be used to overload the assignment (=) operator.

1d(2)

Ans

Operator overloading does not mean changing the function and behavior of that particular operator forever or for the built-in data types. It only facilitates us to perform operations on user-defined data types similar to the ones we do on integral types.

For example if we have 2 integers,

we can assign one to other like this.

```
int a = 5, b = 7;
```

```
a = b;
```

Similarly, we also can also assign one object to another to behave like that

```
my_class obj1(5, "abc"), obj2(10, "xyz");
```

```
obj1 = obj2;
```

Similarly we can use + operator like the above example.

So, we can say that it does not lose its original functionality. It will only change

the behaviour for the specific type. it's overloaded for.

2(a) →

No the statement is wrong.

A derived class can not access the private members of its base class.

A derived class can access all the non-private members of its base class.

Here is a table below:

Access	public	private	protected
Same class	yes	no yes	yes
derived class	yes	No	yes

2(a) or protected inheritance

makes the public and protected members of the base class protected in the derived class

If we want to keep protected all

the public and protected members of base class in derived class then we can use protected access specifier.

2 (b)

```
class A
{
public:
void void f1 ()
{
    cout << "class A" << endl;
}
};
```

```
class B
{
public:
    void f2 ()
    {
        cout << "class B" << endl;
    }
};
```

```
class C
{
public:
    void f3 ()
    {
        cout << "class C" << endl;
    }
};
```

```
class D: public C, public B, public A
```

```
{ public:
```

```
void f4()
```

```
void f4()
```

```
{ cout << "class D" << endl;
```

```
};
```

```
int main()
```

```
{ D d;
```

```
d.f1();
```

```
d.f2();
```

```
d.f3();
```

```
d.f4();
```

```
return 0;
```

```
}
```

output

class A

class B

class C

class D

(2b (or))

virtual base class:-

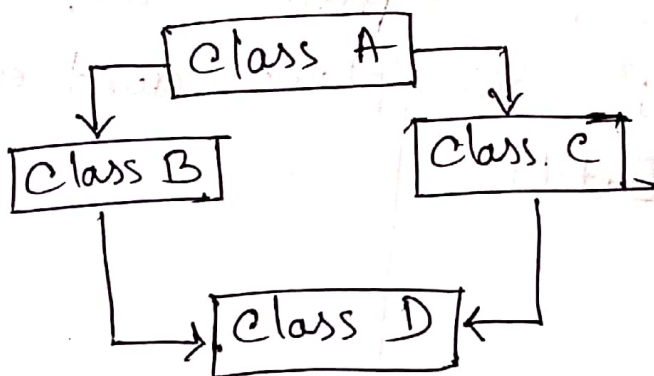
Another names

- * Diamond problems
- * Hybrid inheritance.

virtual base class are used in virtual inheritance in a way of preventing multiple "instances" of a given class appearing in an inheritance hierarchy when using multiple inheritances.

Basically we use virtual base class in hybrid & inheritance.

Suppose we have two derived class B & C and have a common base class A. and also we have another class D that inherits from B & C. Then we can declare the base class A as virtual to ensure that B and C share the same object of A.



Program

```
class A {  
    public:  
    void eat()  
    {  
        cout << "please eat." << endl;  
    }  
}
```

```
class B: virtual public A  
{  
    public:  
    void dontEat()  
    {  
        cout << "please dont eat." << endl;  
    }  
};
```

```
class C: virtual public A  
{  
};
```

```
class D: public B, C  
{  
};
```

```
int main() {  
    D obj;  
    obj.eat();  
    obj.dontEat();  
}
```

Output →

please eat.

please dont eat.

So, there is no problem with ~~the~~ derived class B. It is perfectly called the function `don'tEat()`, which was the property of class B and after that also class D.

2(c) === ? मरिनी

2(D) There is an error in this code. In the main function, "`obc.cheers();`" line has a mistake. If we call this `cheers()` function ^{like this way} ~~x~~ then there will be an ambiguity. As we have a function with the same name in class A and B. We can solve this ambiguity by calling this function like this way.

`obc.A::cheers();`
`obc.B::cheers();`

output

class A: Hip-hip-hooray
class B: Hip-hip hooray

3(a)

Virtual Function:-

A virtual function is a member function that is declared within a base class and redefined by a derived class. To create a virtual function, precede the function's declaration with the keyword "virtual". When a virtual function is redefined by a derived class, the keyword virtual is not needed.

Example →

```
Class A // Base class
{
    public:
        virtual void show()
        { cout << "Base" << endl; }
};

Class B: public A // derived 1
{
    public:
        void show()
        { cout << "Class B" << endl; }
};
```

```
class C : public A // derived 2'
```

```
{ public:
```

```
void show ()
```

```
{ cout << "class C" << endl; }
```

```
};
```

```
int main ()
```

```
{ A B obj1;
```

```
  C obj2;
```

```
  A * ptr;
```

```
  ptr = &obj1;
```

```
  ptr -> show();
```

```
  ptr = &obj2;
```

```
  ptr -> show();
```

```
}
```

3(b)

3(b) Early Binding If the compiler ^{knows} ~~knows~~ at the compile time which function is called it is called early binding.

OR

Early binding refers to those events that ~~must occur at run time~~ can be known at compile time. Specifically it refers to those functions calls that can be resolved during compilation

Advantage The main advantage of early binding is that it is very efficient.

disadvantage It has lack of flexibility.

Late binding → If a compiler does not know at compile time which functions to call up until the run time, it is called late binding.

Advantage → Flexibility at run-time.

Disadvantage → There is more overhead associated with a function call. It makes function calls slower than early binding.

3 (c)

```
#include <bits/stdc++.h>
using namespace std;

class A { // Base class
public:
    virtual void print()
    { cout << "Base function called" << endl; }
};
```

```
class B: public A { // derived class
```

```
public:
```

```
void print() {
```

```
    cout << "derived class called" << endl;
```

```
}
```

```
};
```

```
int main() {
```

```
    B
```

```
    obj;
```

```
    A* base = &obj; // pointer of A type that  
                    // points to obj,
```

```
    base -> print(); // calls member function  
                    // of derived (B) class
```

```
    return 0;
```

```
}
```

4a

4(a)

Exception, when we write a simple C++ program, the first thing we see when a program doesn't work correctly those known as bugs. Most common bugs are logical ~~error~~ error or syntax error. We can overcome those bugs by debugging but sometimes we come across some problems other than logical or syntax error those known as exceptions.

exception handling mechanism

- Extra {
- ① Finding the problem (Hit the exception)
 - ② infor an error is occurred (Throw)
 - ③ Receive error information (Catch the exception)
 - ④ correct it (Handle the exception)

Advantage ① * Remove error-handling code from the software's main line of code.

② Separating error-handling code from 'Regular' code.

4(b)

```
template <typename T>
T min(T n1, T n2)
{
    T new;
    if (n1 > n2)
        new = n2;
    else
        new = n1;
    return new;
}
```

```
int main ()
```

```
{
    int A = 10, B = 20, C;
```

```
    C = min(A, B);
```

```
    cout << "Minimum value is:" << C << endl;
```

Output

Minimum value is: 10

4(c)

generic function

A generic function that represent several function performing same task but on different data types is called function template.

For example if we want to create a function that to add two integers and a float then we have to write two function separately, one accept integer type and another accept float.

But if we use function template / generic function then we can perform both addition (between integer and float) by a single function. which is function template.

Example `template <typename T>`

~~Generic class~~

```
T name_of_fun(T variable_name)
{
    body
}
```

Generic Class :-

Same as function template / generic function we can also create class templates for generic class.

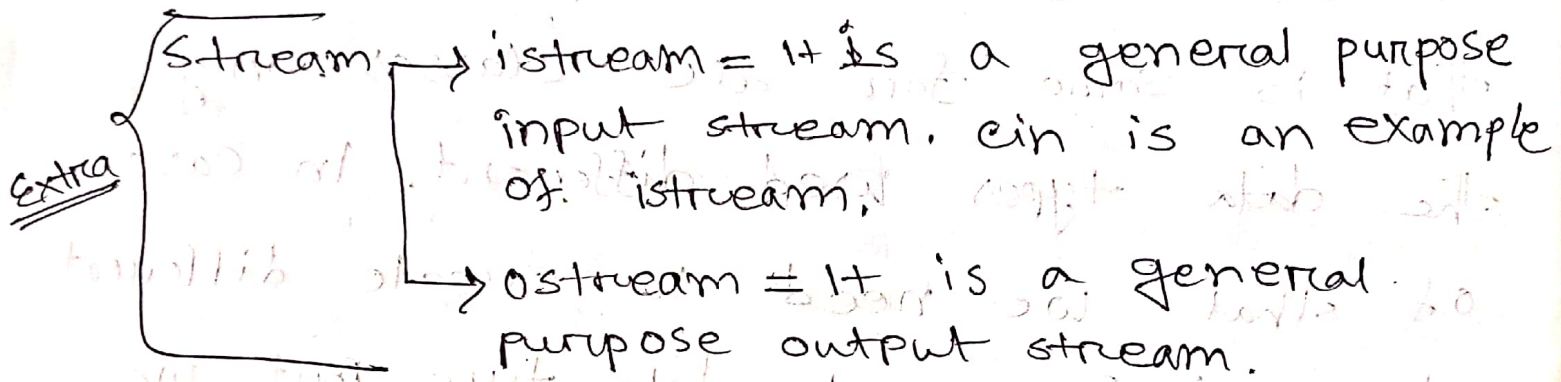
If we need a class implementation that is same for all classes; only the data types used different. In case of that we need to create different class ~~for~~ for each data type. But we can solve it by creating a single generic class. which can be used for different data types.

Example

```
template < class T >
class class_Name
{
    public:
        T variable;
        T sum (T arg);
        ...
};
```

5(a)

Stream A C++ stream is a flow of data into or out of a program. Such as the data written to cout or read from cin.



Stream classes:-

5(b)

width()

```
cout << "Implementing ios::width" << endl;
```

```
char c = 'A';
```

```
cout.width(5); // Adjusting width will be 5
```

```
cout << c << endl;
```

```
int temp = 10;
```

```
cout << temp;
```

output

Implementing ios::width

A

10

precision()

```
float a = 3.1422;
```

```
cout.setf(ios::fixed); → setf on next page
```

```
cout.precision(2);
```

```
cout << a;
```

output

3.14

fill()

```
cout << "Implementing ios::fill" << endl;
```

```
char ch = 'a';
```

```
cout.fill('*');
```

```
cout.width(10);
```

```
cout << ch << endl;
```

```
int i = 1;
```

```
cout.width(5);
```

```
cout << i;
```

output

```
*****a
```

```
****a
```

setf()

```
cout << "Implementing ios::setf" << endl;
```

```
int value1 = 100, value2 = 200;
```

```
cout.setf(ios::showpos);
```

```
cout << value1 << " " << value2;
```

output

```
Implementing ios::setf()
```

```
+100 +200
```

5c

5c

```
#include <bits/stdc++.h>
using namespace std;
```

```
class student {
```

```
public:
```

```
    string name, semester, CourseCode,
    co-title;
```

```
};
```

```
int main()
```

```
class obj student obj
```

```
ofstream file_Name;
```

```
String In-Name, In-Sem, In-CourseCode,
In-co-title;
```

```
file_Name.open("who Are You.txt", ios::out);
```

```
cout << "Name:" << endl;
```

```
cin >> In-Name;
```

```
file_Name << In-Name << endl;
```

```
cout << " semester:" << endl;
```

```
cin >> In-Sem;
```

```
file_Name << In-Sem << endl;
```

```
cout << " course code:" << endl;
```

```
cin >> In-courseCode;
```

```
file_Name << In-CourseCode << endl;
```

N.B: Here you can skip the class portion. You can start from main fun.

cout << "course Title:" << endl;

cin >> In_CoTitle;

file_Name << In_CoTitle << endl;

Thank You ☺